

Evaluation of IPC through Linux Pipes

Patrick DeCabooter and Shaw Liu

1 Introduction

The UNIX pipe is the oldest IPC mechanism still in daily use, and its cost model is usually taken on faith. The buffer is sixteen pages, writers block when it fills, and larger writes amortize better. We measured those claims and then asked where the time goes. The dominant variable turns out to be placement and not payload size.

2 Apparatus and method

We ran everything on an Ubuntu 24.04 guest with a custom Linux 7.2.4 kernel on arm64, eight vCPUs and 12 GiB of RAM, under QEMU with hvf on an Apple M1. The page size is 4 KiB and pipe-max-size is 1 MiB, so the resize sweep runs unprivileged. Absolute figures include hypervisor overhead, but the placement comparisons do not depend on it.

We place the processes with `sched_setaffinity` after the fork, since `taskset` cannot express parent on core 0 and child on core 1. Each configuration runs five times and we take the minimum, since noise can only add time. Throughput is a rate, so we report the maximum and never a mean.

3 Choosing a clock

Variables. The cause is the clock API. The effects are accuracy and granularity. **Hypothesis.** `clock_gettime` can resolve a single exchange and `gettimeofday` cannot.

We compared `clock_gettime(CLOCK_MONOTONIC)`, `gettimeofday`, and `cntvct_el0`, which is this architecture’s analogue of `rdtsc`. Timing `sleep(5)`, all three recovered 5.0024s and agreed within 53 ppm, so all three are accurate. We then read each clock 10^6 times back to back. The quantum is 41 ns

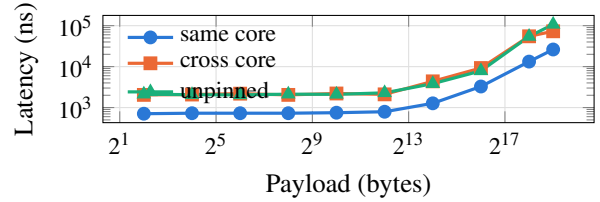


Figure 1: One-way latency, taken as half a round trip, minimum of five runs.

for `clock_gettime`, 41.67 ns for `cntvct_el0`, and 1 μ s for `gettimeofday`, and 97.9% of adjacent `gettimeofday` readings are identical. A clock that cannot separate two of its own calls cannot measure a 708 ns exchange, so we discarded it. We chose `clock_gettime`, because it shares the quantum of the raw counter and is portable. Neither cycle counter caveat bites here, since the clock is frequency invariant and reads within 83 ns across cores.

4 Latency

Variables. The causes are payload size and placement. The effect is one-way latency, taken as half a round trip so that only one clock is involved. **Hypothesis.** Flat while the fixed cost dominates, then linear once the copying dominates.

Figure 1 shows that shape. Below 4 KiB every curve is flat, so the cost is per exchange and not per byte. Above 16 KiB the growth is linear. The part we did not expect is the constant offset in the flat region. Same-core costs 708 ns and cross-core costs 2042 ns, a penalty of 1333 ns that does not depend on the payload. Unpinned runs track the cross-core curve, so the scheduler spreads the pair by default.

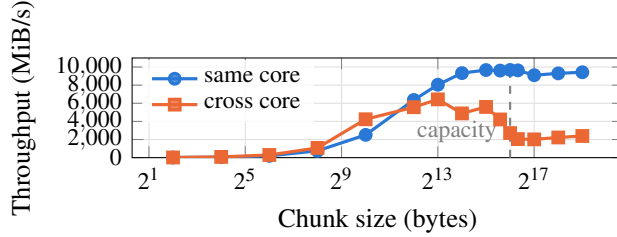


Figure 2: Streaming throughput, best of five runs. The dashed line is the default 64 KiB capacity.

5 Throughput

Variables. The causes are chunk size and placement. The effect is sustained one-way bandwidth, acknowledged once for the whole transfer. **Hypothesis.** Throughput rises as the fixed per-write cost amortizes, then falls once the chunks exceed the capacity.

Figure 2 shows that the first half is right and the second half is wrong. Only the cross-core curve collapses, from 6431 MiB/s at 8 KiB to 2391 MiB/s at 512 KiB. The same-core curve climbs to 9678 MiB/s and stays flat through chunks eight times the capacity. This rules out blocking as the explanation, because the pipe is the same size in both cases, so the same-core writer blocks just as often. What is lost is overlap, and Section 6 accounts for it.

6 Where the time goes

Variables. The causes are placement and the ratio of payload to capacity. The effects are the split of elapsed time between user, kernel and blocked, and the context switches per exchange. **Hypothesis.** If the cross-core penalty is a scheduling cost, both placements should issue the same system calls and the same context switches, and the difference should appear as time that is neither user nor kernel.

A nonblocking pipe accepts exactly 65,536 bytes before EAGAIN, which matches F_GETPIPE_SZ and the sixteen page folklore. Table 1 accounts for the wall clock with getrusage. At 4 B the same-core pair spends almost all its time in the kernel and none of it blocked. The cross-core pair does the same work, and strace confirms two write and two read calls per round trip, but it is ten times slower, and 61 % of its wall clock is neither user nor system time. That

Payload	Place	μ s/RT	%usr	%sys	%blk	cs/RT
4 B	same	2.3	0	101	0	1.99
4 B	cross	23.0	3	37	61	2.00
512 KiB	same	58.9	14	88	0	15.99
512 KiB	cross	346.8	0	53	47	15.99

Table 1: Where the wall clock goes, per round trip. Percentages are getrusage totals for both processes against elapsed time, and “cs/RT” is voluntary context switches.

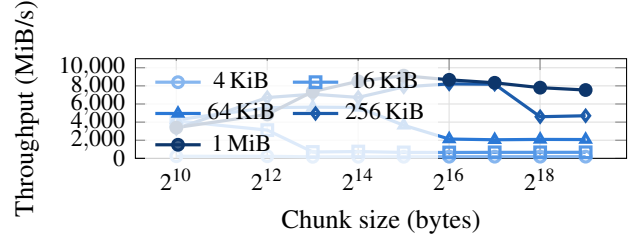


Figure 3: Cross-core throughput with the pipe capacity set by F_SETPIPE_SZ, one curve per capacity, light to dark.

residue is the 1333 ns penalty, because it is time spent descheduled while waiting for a wakeup on another core.

The context switch column connects this to the capacity. There are 1.99 voluntary switches per round trip at 4 B and 15.99 at 512 KiB, which is exactly $2 \lceil \text{size/capacity} \rceil$ against the measured 64 KiB. Figure 3 then makes the capacity the independent variable, and the knee follows the configured size, so the correlation is causal.

7 Conclusions

Pipe cost is set by two quantities, and the payload size is only one of them. The capacity fixes how many handoffs a transfer needs, and the placement fixes the price of each one. So spreading a pair across cores only helps while the messages stay below the capacity, and past that point the pair is four times faster on a single core. Everything ran virtualized, so the absolute latencies carry hypervisor overhead, and no PMU was available in the guest, so Table 1 rests on getrusage and strace.

8 Use of AI tools

interpretation and the claims are our own.

We used an AI coding assistant to review the benchmark code and run the sweeps, but the design, the